

Bilan de la tâche 4

Precision auto-tuning and verified computing

Réunion du projet Interflop

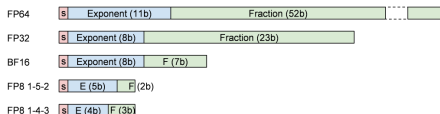
LIP6, Sorbonne Université

20 mars 2025

Axes de recherche

- 1 Algorithmes en précision mixte
- 2 Analyse/estimation de la fiabilité numérique
- 3 Amélioration de la précision des algorithmes numériques
- 4 Formats numériques compressés

Différents formats flottants



↘ précision :

- ↘ temps de calcul 😊
- ↘ volume de résultats échangés 😊
- ↗ efficacité énergétique 😊

Compromis performance / fiabilité :

- tirer parti des précisions faibles
- fournir des résultats de précision satisfaisante

⇒ calculs en précision mixte

Comment sélectionner la précision adéquate pour chaque variable/opération ?

• Auto-ajustement de la précision

transformer un code source en précision mixte, quels que soient les algorithmes implantés

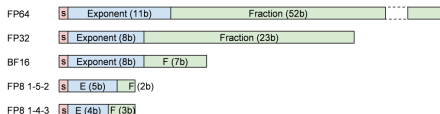
thèse de Quentin Ferro

• Algorithmes en précision adaptative

dans un algorithme, sélectionner de manière adaptative la précision minimale pour chaque calcul

stage (et thèse) de Roméo Molina

Différents formats flottants



↘ précision :

- ↘ temps de calcul 😊
- ↘ volume de résultats échangés 😊
- ↗ efficacité énergétique 😊

Compromis performance / fiabilité :

- tirer parti des précisions faibles
- fournir des résultats de précision satisfaisante

⇒ calculs en précision mixte

Comment sélectionner la précision adéquate pour chaque variable/opération ?

• Auto-ajustement de la précision

transformer un code source en précision mixte, quels que soient les algorithmes implantés

thèse de Quentin Ferro

• Algorithmes en précision adaptative

dans un algorithme, sélectionner de manière adaptative la précision minimale pour chaque calcul

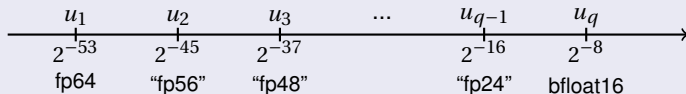
stage (et thèse) de Roméo Molina

Produit matrice-vecteur creux en précision adaptative

Principe

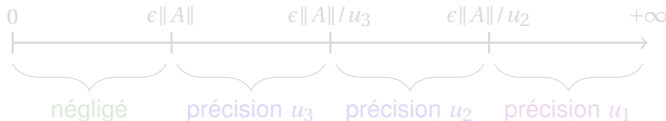
Calcul de $y = Ax$ avec une précision souhaitée ϵ

$\Rightarrow y = \sum_{k=1}^q A_k x$ calculé avec q précisions de travail



La précision sélectionnée dépend de chaque élément a_{ij} :

$$\begin{cases} |a_{ij}| \leq \epsilon \|A\| & \Rightarrow a_{ij} \text{ négligé} \\ |a_{ij}| \in [\epsilon \|A\| / u_{k+1}, \epsilon \|A\| / u_k) & \Rightarrow \text{précision } u_k \\ |a_{ij}| > \epsilon \|A\| / u_2 & \Rightarrow \text{précision } u_1 \end{cases}$$



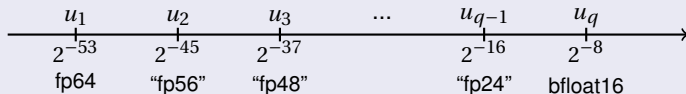
Théorème : le vecteur calculé $\hat{y}$ vérifie $\|\hat{y} - y\| / \|A\| \|x\| \leq c\epsilon$

Produit matrice-vecteur creux en précision adaptative

Principe

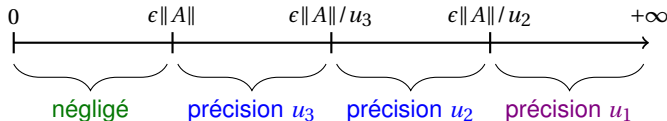
Calcul de $y = Ax$ avec une précision souhaitée ϵ

$\Rightarrow y = \sum_{k=1}^q A_k x$ calculé avec q précisions de travail



La précision sélectionnée dépend de chaque élément a_{ij} :

$$\begin{cases} |a_{ij}| \leq \epsilon \|A\| & \Rightarrow a_{ij} \text{ négligé} \\ |a_{ij}| \in [\epsilon \|A\| / u_{k+1}, \epsilon \|A\| / u_k) & \Rightarrow \text{précision } u_k \\ |a_{ij}| > \epsilon \|A\| / u_2 & \Rightarrow \text{précision } u_1 \end{cases}$$



Théorème : le vecteur calculé $\hat{y}$ vérifie $\|\hat{y} - y\| / \|A\| \|x\| \leq c \epsilon$

- 33 matrices de la collection SuiteSparse¹ et de partenaires industriels
 - matrices avec du potentiel : valeurs flottantes, magnitudes différentes
 - applications variées : dynamique des fluides, semiconducteurs...
 - matrices carrées de taille 10^5 à 10^6
 - avec 10^6 à 10^8 éléments non-nuls
- cibles d'incertitude :
 - $\epsilon = 2^{-24}$ (équivalent à fp32)
 - $\epsilon = 2^{-37}$ (pas d'équivalent)
 - $\epsilon = 2^{-53}$ (équivalent à fp64)

¹ <https://sparse.tamu.edu/>

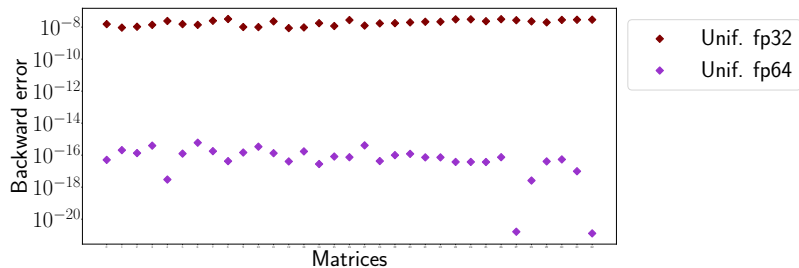
- 33 matrices de la collection SuiteSparse¹ et de partenaires industriels
 - matrices avec du potentiel : valeurs flottantes, magnitudes différentes
 - applications variées : dynamique des fluides, semiconducteurs...
 - matrices carrées de taille 10^5 à 10^6
 - avec 10^6 à 10^8 éléments non-nuls
- cibles d'incertitude :
 - $\epsilon = 2^{-24}$ (équivalent à fp32)
 - $\epsilon = 2^{-37}$ (pas d'équivalent)
 - $\epsilon = 2^{-53}$ (équivalent à fp64)

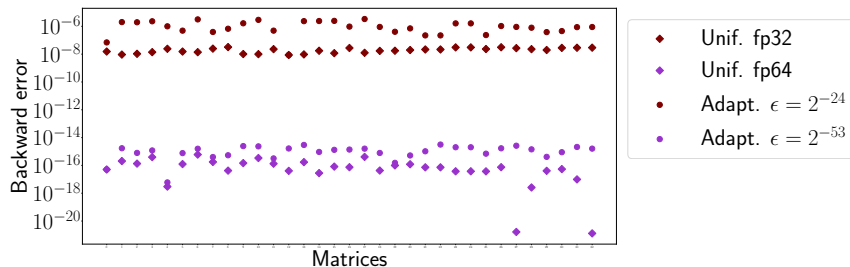
¹ <https://sparse.tamu.edu/>

Différents ensembles de formats :

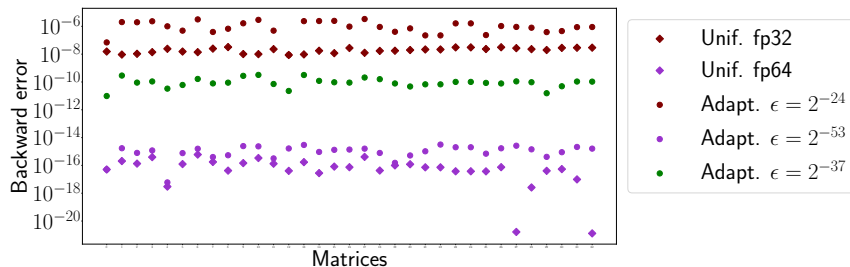
- 2 **précisions** : fp32, fp64
- 3 **précisions** : bfloat16, fp32, fp64
- 7 **précisions** : bfloat16, "fp24", fp32, "fp40", "fp48", "fp56", fp64

	Bits	
	Mantissa	Exponent
bfloat16	8	8
"fp24"	16	8
fp32	24	8
"fp40"	29	11
"fp48"	37	11
"fp56"	45	11
fp64	53	11





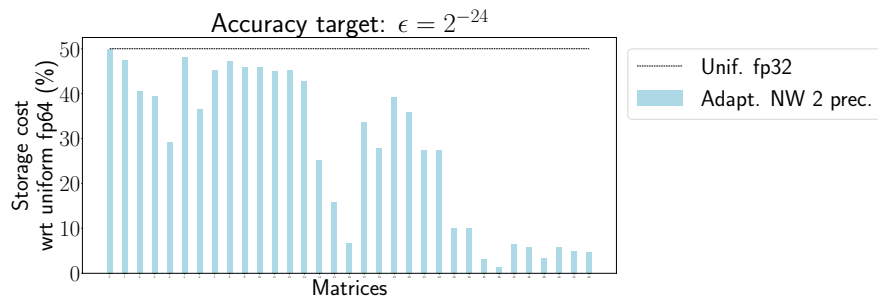
Les méthodes adaptatives permettent de cibler des niveaux d'erreur similaires à ceux obtenus en précision uniforme



... et des niveaux d'incertitude intermédiaires !

Produit matrice-vecteur creux en précision adaptative

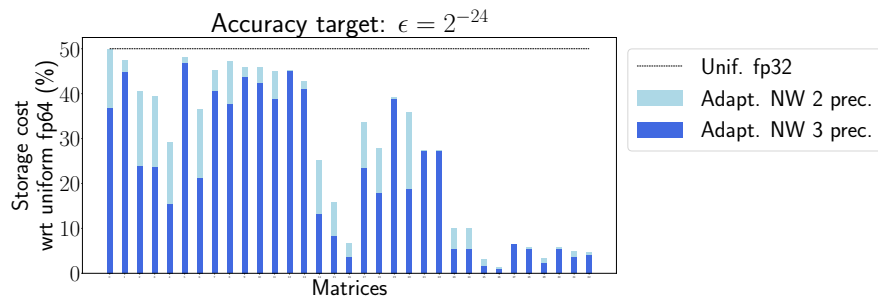
Performances



- Gain en stockage jusqu'à $\times 36$
- Gain en temps jusqu'à $\times 7$

Produit matrice-vecteur creux en précision adaptative

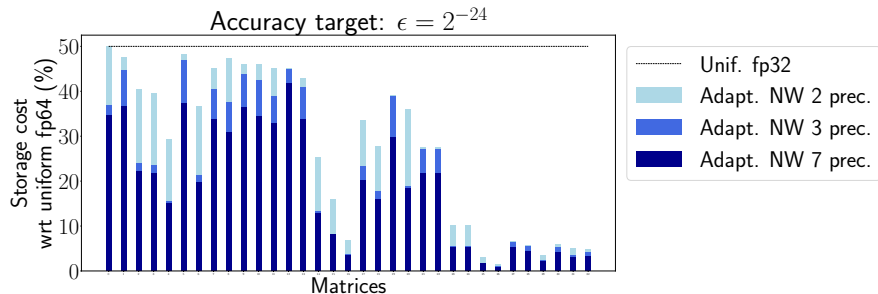
Performances



- Gain en stockage jusqu'à $\times 36$
- Gain en temps jusqu'à $\times 7$

Produit matrice-vecteur creux en précision adaptative

Performances



- Gain en stockage jusqu'à $\times 36$
- Gain en temps jusqu'à $\times 7$

Observation: Adaptive SpMV **sorts elements by magnitude**

- ⇒ No need to represent all magnitudes in each bucket
- ⇒ No need for 11 bits to store the exponent
- ⇒ If the difference in magnitude is less than 2^8 within a bucket, **a 3-bit exponent is sufficient!**
- ⇒ The sign bit can also be removed by using separate buckets for positive and negative elements
- ⇒ Development of RPRE (Reduced Precision and Reduced Exponent) and RPREU (Reduced Precision and Reduced Exponent, Unsigned) formats

Observation: Adaptive SpMV **sorts elements by magnitude**

- ⇒ No need to represent all magnitudes in each bucket
- ⇒ No need for 11 bits to store the exponent
- ⇒ If the difference in magnitude is less than 2^8 within a bucket, **a 3-bit exponent is sufficient!**
- ⇒ The sign bit can also be removed by using separate buckets for positive and negative elements
- ⇒ Development of RPRE (Reduced Precision and Reduced Exponent) and RPREU (Reduced Precision and Reduced Exponent, Unsigned) formats

Observation: Adaptive SpMV **sorts elements by magnitude**

- ⇒ No need to represent all magnitudes in each bucket
- ⇒ No need for 11 bits to store the exponent
- ⇒ If the difference in magnitude is less than 2^8 within a bucket, **a 3-bit exponent is sufficient!**
- ⇒ The sign bit can also be removed by using separate buckets for positive and negative elements
- ⇒ Development of RPRE (Reduced Precision and Reduced Exponent) and RPREU (Reduced Precision and Reduced Exponent, Unsigned) formats

Observation: Adaptive SpMV **sorts elements by magnitude**

- ⇒ No need to represent all magnitudes in each bucket
- ⇒ No need for 11 bits to store the exponent
- ⇒ If the difference in magnitude is less than 2^8 within a bucket, **a 3-bit exponent is sufficient!**
- ⇒ The sign bit can also be removed by using separate buckets for positive and negative elements
- ⇒ Development of RPRE (Reduced Precision and Reduced Exponent) and RPREU (Reduced Precision and Reduced Exponent, Unsigned) formats

Observation: Adaptive SpMV **sorts elements by magnitude**

- ⇒ No need to represent all magnitudes in each bucket
- ⇒ No need for 11 bits to store the exponent
- ⇒ If the difference in magnitude is less than 2^8 within a bucket, **a 3-bit exponent is sufficient!**
- ⇒ The sign bit can also be removed by using separate buckets for positive and negative elements
- ⇒ Development of RPRE (Reduced Precision and Reduced Exponent) and RPREU (Reduced Precision and Reduced Exponent, Unsigned) formats

Observation: Adaptive SpMV **sorts elements by magnitude**

- ⇒ No need to represent all magnitudes in each bucket
- ⇒ No need for 11 bits to store the exponent
- ⇒ If the difference in magnitude is less than 2^8 within a bucket, **a 3-bit exponent is sufficient!**
- ⇒ The sign bit can also be removed by using separate buckets for positive and negative elements
- ⇒ Development of RPRE (Reduced Precision and Reduced Exponent) and RPREU (Reduced Precision and Reduced Exponent, Unsigned) formats

Adaptive SpMV performance

Gains depending on the matrices, up to:

- 99% data reduction
- 96% execution time reduction

Application to Krylov solvers

- Adaptive SpMV with a reasonable uncertainty target preserves convergence
- The uncertainty level can be tuned to find the best trade-off between iteration cost and convergence speed

[SISC 2024, EuroPar 2024]

Développements dans PROMISE

...qui utilise l'Arithmétique Stochastique Discrète via CADNA

thèse de Quentin Ferro (soutenue en oct. 2024)

Classic arithmetic

$$A \oplus B \longrightarrow R$$

$R = 3.14237654356891$

DSA

Random
rounding

$$A_1 \oplus B_1 \xrightarrow{\text{die}} R_1$$

$$A_2 \oplus B_2 \xrightarrow{\text{die}} R_2$$

$$A_3 \oplus B_3 \xrightarrow{\text{die}} R_3$$

$R_1 = \mathbf{3.141}354786390989$

$R_2 = \mathbf{3.143}689456834534$

$R_3 = \mathbf{3.142}579087356598$

- each operation executed 3 times with a random rounding mode

Classic arithmetic

$$A \oplus B \longrightarrow R$$

 $R = 3.14237654356891$

DSA

Random
rounding

$$A_1 \oplus B_1 \xrightarrow{\text{die}} R_1$$

$$A_2 \oplus B_2 \xrightarrow{\text{die}} R_2$$

$$A_3 \oplus B_3 \xrightarrow{\text{die}} R_3$$

 $R_1 = \mathbf{3.141}354786390989$
 $R_2 = \mathbf{3.143}689456834534$
 $R_3 = \mathbf{3.142}579087356598$

- each operation executed 3 times with a random rounding mode
- number of correct digits in the results estimated using Student's test with the confidence level 95%

Classic arithmetic

$$A \oplus B \longrightarrow R$$

$R = 3.14237654356891$

DSA

Random
rounding

$$A_1 \oplus B_1 \xrightarrow{\text{die}} R_1$$

$$A_2 \oplus B_2 \xrightarrow{\text{die}} R_2$$

$$A_3 \oplus B_3 \xrightarrow{\text{die}} R_3$$

$R_1 = \mathbf{3.141}354786390989$

$R_2 = \mathbf{3.143}689456834534$

$R_3 = \mathbf{3.142}579087356598$

- each operation executed 3 times with a random rounding mode
- number of correct digits in the results estimated using Student's test with the confidence level 95%
- operations executed synchronously
 - ⇒ detection of numerical instabilities
Ex: if (A>B) with A-B numerical noise
 - ⇒ optimization of stopping criteria

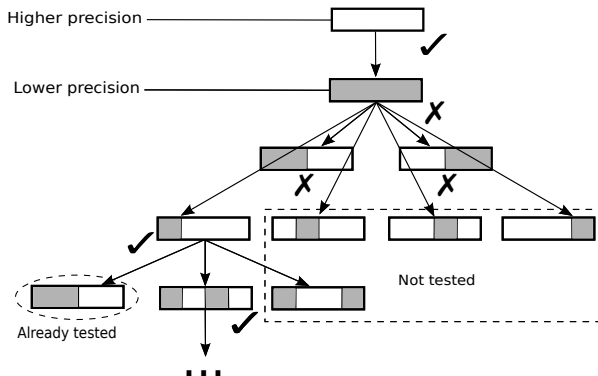


- implements Discrete Stochastic Arithmetic for **C/C++** or **Fortran** codes
- provides **stochastic types** (3 floating-point variables and an integer)
- all operators and mathematical functions overloaded
⇒ **few modifications in user programs**
- support for **MPI**, **OpenMP**, **GPU** codes
- in **one CADNA execution**: accuracy of any result, complete list of numerical instabilities

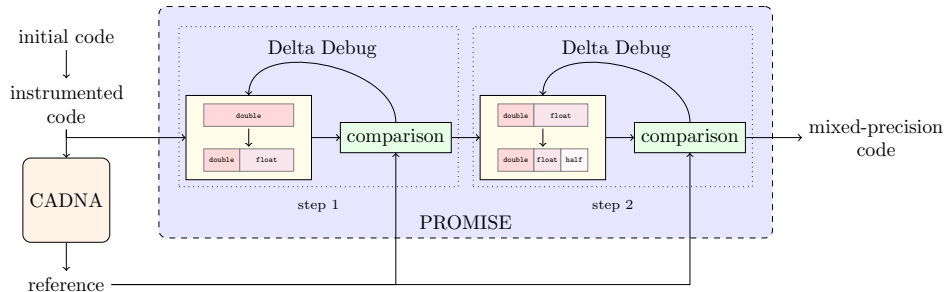
[Chesneaux'90], [Jézéquel & al'08], [Lamotte & al'10], [Eberhart & al'18],...



- Transformation of C/C++ codes into mixed precision based on a required result accuracy
- Search for a suitable type configuration
 - Validated with CADNA
 - Delta Debug: average complexity of $O(n \log n)$ for n variables



PROMISE data flow in double, single and half precision



- step 1: code in double → variables relaxed to single precision
- step 2: single precision variables → variables relaxed to half precision

Precision autotuning of neural networks

- neural networks created and trained with Keras or PyTorch
- automatically transformed into C++ codes to be used with PROMISE

Sine NN: interpolation network approximating the sine function

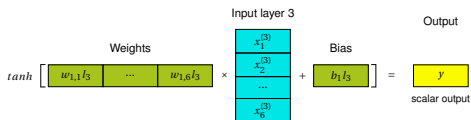
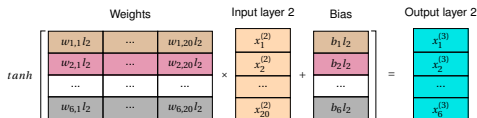
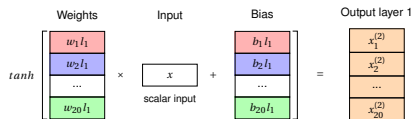
- Scalar input/output
- 3 dense layers with tanh activation function:

- 20 neurons $\rightarrow$ 21 types to set
- 6 neurons $\rightarrow$ 7 types to set
- 1 neuron $\rightarrow$ 2 types to set

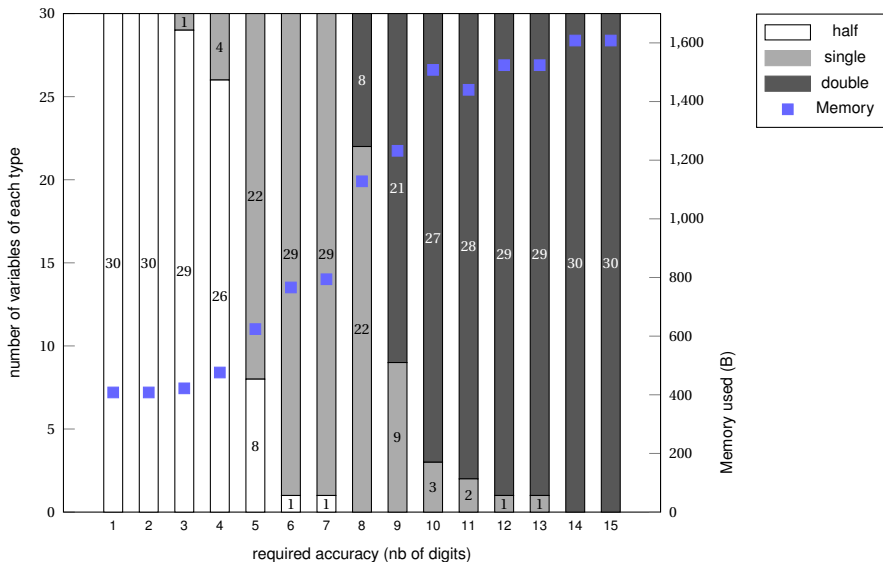
$\Rightarrow$ 30 types to set in total

2 approaches:

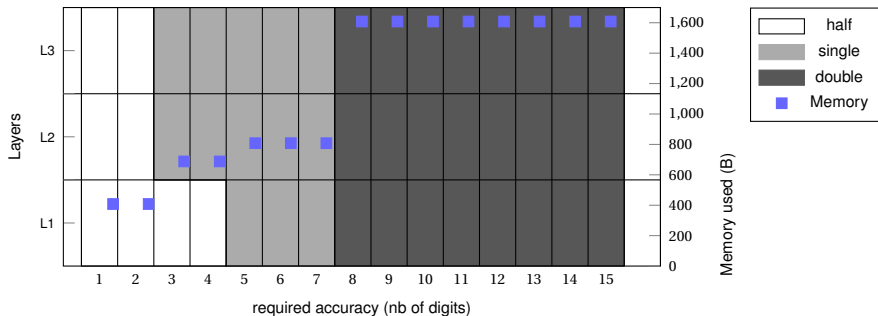
- one type per neuron
- one type per layer



Sine NN, one type per neuron



Sine NN, one type per layer



In this talk, input=0.5
similar trends observed with different input values

Classification of handwritten digits:

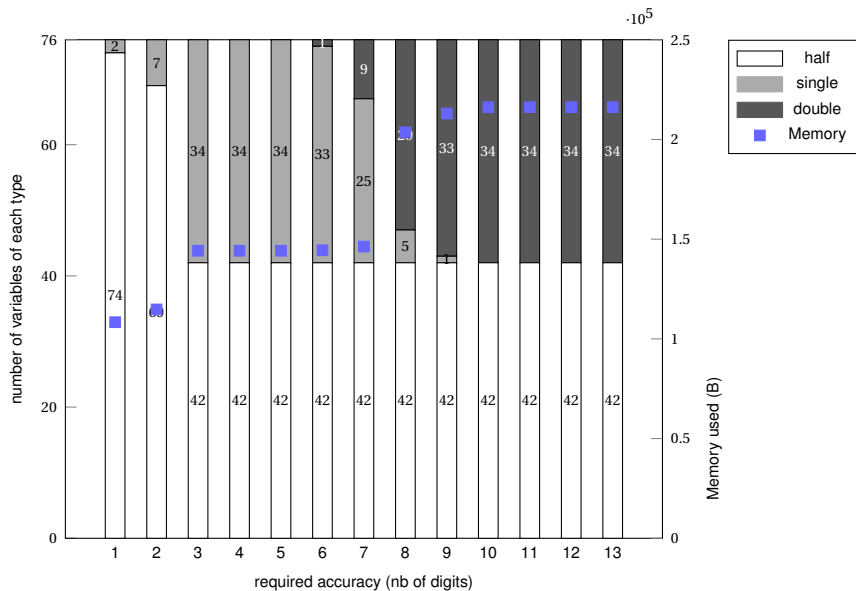
- input: vector of size 784 (flatten image)
- output : vector of size 10, probability distribution for the 10 different classes
- 2 dense layers:
 - 64 neurons and ReLU activation function
→ 65 types to set
 - 10 neurons and softmax activation function
→ 11 types to set



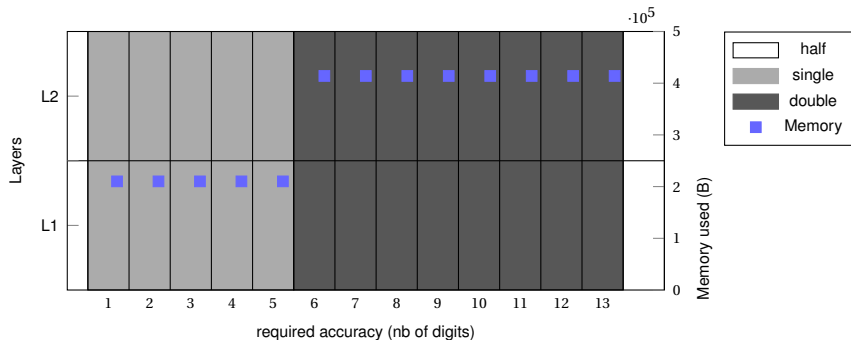
wikipedia.org

⇒ 76 types to set in total

MNIST NN, one type per neuron



MNIST NN, one type per layer

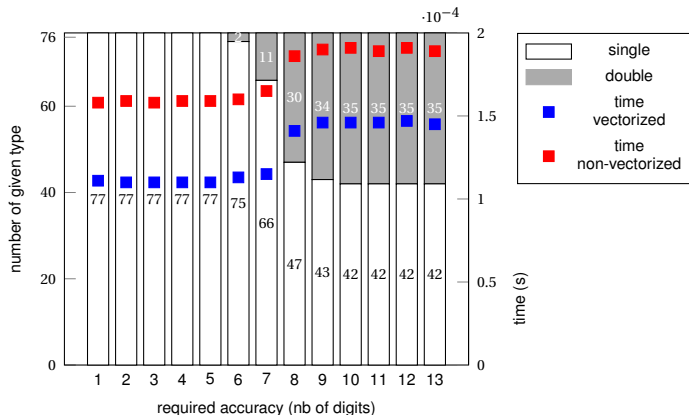


In this talk, input image `test_data[61]` from MNIST data base
similar trends observed with different input images

Time gain with MNIST NN, one type per neuron

matrix-vector products vectorized using OpenMP SIMD on AVX2

⇒ 8 fp32 (or 4 fp64) operations in parallel

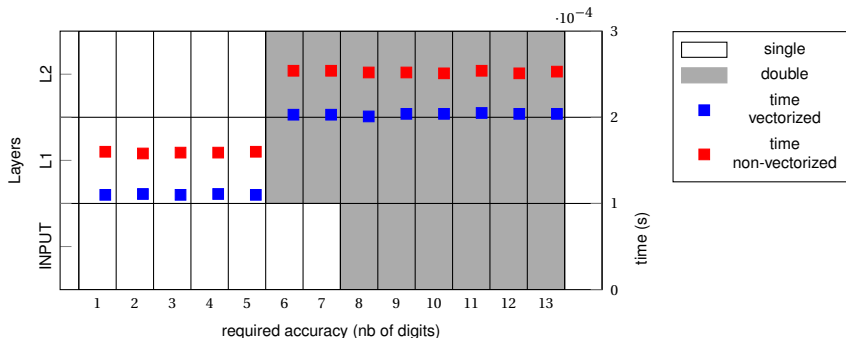


time ratio fp64 / mixed precision:

- up to 1.30 for non-vectorized codes
- up to 1.41 for vectorized codes

theoretical time ratio fp64 / fp32 = 2

Time gain with MNIST NN, one type per layer



time ratio fp64 / fp32:

- up to 1.60 for non-vectorized codes
- up to 1.86 for vectorized codes

time ratio non-vectorized / vectorized:
up to 1.45

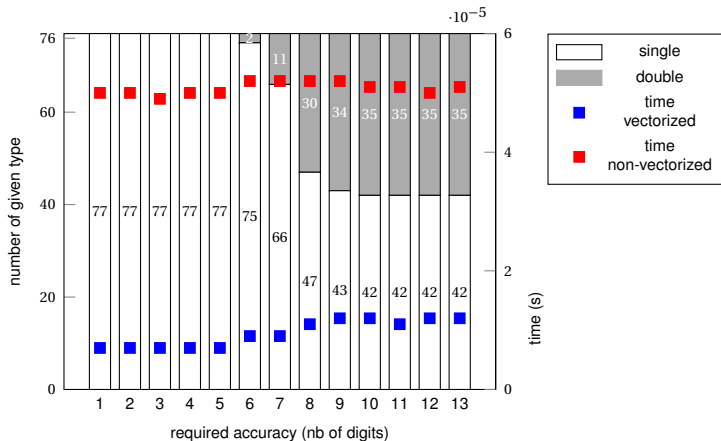
theoretical time ratio:

$$\text{fp64} / \text{fp32} = 2$$

$$\text{non-vect.} / \text{vect.} = 8 \text{ in fp32, } 4 \text{ in fp64}$$

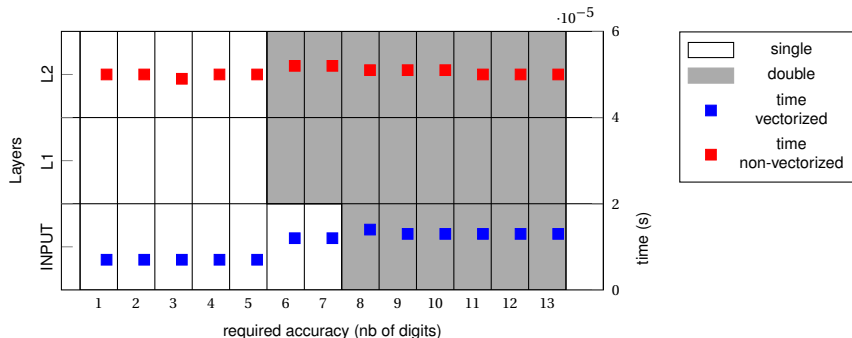
Matrix-vector products in MNIST NN

one type per neuron



Matrix-vector products in MNIST NN

one type per layer



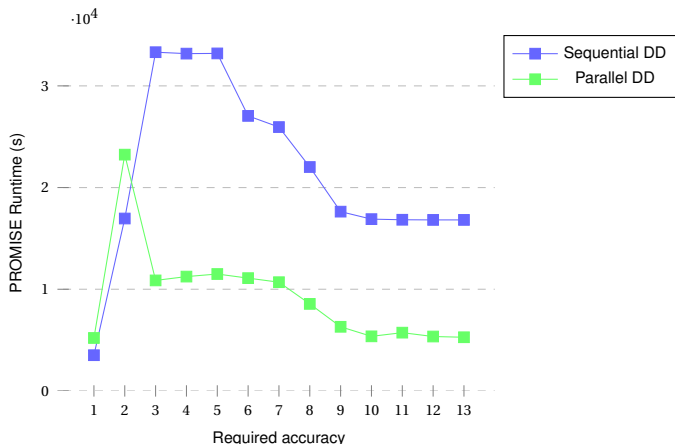
time ratio fp64 / fp32:

- no time difference for non-vectorized codes
- 2 for vectorized codes

time ratio non-vectorized / vectorized: up to 7.2 in fp32 and 3.9 in fp64

😊 close to theoretical ratio

Execution time of PROMISE for MNIST NN



Parallel version of the Delta Debug from [Hodován & Kiss'16]:
type configurations on the same level are tested in parallel

Speedup up to 3.2 on a CPU having 6 cores with 16 GB RAM

[NSV 2022, MCSoc 2023]

- Based on the LLVM LibTooling library through Clang compilation
- Instrumentation for CADNA: replaces the old non-robust, *ad hoc*, and incomplete instrumentation tool
 - + instrumentation for MPI codes
- Instrumentation for PROMISE: replaces the tedious manual instrumentation and the *ad hoc* and incomplete code parsing performed in PROMISE

Listing: Code source

```
#include <stdio.h>

using namespace std;

int main() {
    double x = 42;
    double y = 193;
    double res;

    res = 4*x*x+9*y*y*y;

    printf("res: %.2e\n", res);

    return 0;
}
```

Listing: Code instrumenté pour CADNA

```
#include <cadna.h>
#include <stdio.h>

using namespace std;

int main() {
    cadna_init(0);
    double_st x = 42;
    double_st y = 193;
    double_st res;

    res = 4*x*x+9*y*y*y;

    printf("res: %s\n", strp(res));

    cadna_end();
    return 0;
}
```

Listing: Code MPI instrumenté pour CADNA

```
int main(int argc, char *argv[]) {
    cadna_mpi_init(p, -1);

    ...

    MPI_Send( &res, 1, MPI_DOUBLE_ST, 0, 0, MPI_COMM_WORLD);

    ...

    MPI_Recv( &tmp, 1, MPI_DOUBLE_ST, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &status);

    ...

    cadna_mpi_end();
    MPI_Finalize();
    return 0;
}
```

Listing: Code source

```
#include <stdio.h>
#include <promise.h>

using namespace std;

int main() {
    double x = 42;
    double y = 193;
    double res;

    res = 4*x*x+9*y*y*y;

    printf("res: %.2e\n", res);

    PROMISE_CHECK_VAR(res);

    return 0;
}
```

Listing: Code instrumenté pour PROMISE

```
#include <stdio.h>
#include <promise.h>

using namespace std;

int main() {
    __PR_0__ x = 42;
    __PR_1__ y = 193;
    __PR_2__ res;

    res = 4*x*x+9*y*y*y;

    printf("res: %.2e\n", res);

    PROMISE_CHECK_VAR(res);

    return 0;
}
```

Listing: Code source

```
#include <stdio.h>
#include <promise.h>

using namespace std;

int main() {
    PROMISE_START();
    double x = 42;
    double y = 193;
    PROMISE_END();

    double res;

    res = 4*x*x+9*y*y*y;

    printf("res: %.2e\n", res);

    PROMISE_CHECK_VAR(res);

    return 0;
}
```

Listing: Code instrumenté pour PROMISE

```
#include <stdio.h>
#include <promise.h>

using namespace std;

int main() {
    PROMISE_START();
    __PR_0__ x = 42;
    __PR_1__ y = 193;
    PROMISE_END();

    double res;

    res = 4*x*x+9*y*y*y;

    printf("res: %.2e\n", res);

    PROMISE_CHECK_VAR(res);

    return 0;
}
```

- Outil d'instrumentation : nettoyage / merge git
- Extension de PROMISE à la précision arbitraire avec FloatX: <https://github.com/oprecomp/FloatX>
- Extension de PROMISE aux GPU
- Autres critères dans PROMISE (consommation d'énergie ?)
- Auto-ajustement de la précision dans les réseaux de neurones : entraînement ?
- Auto-ajustement de la précision dans différents domaines applicatifs

postdoc de Marek Felšöci avec THALES

postdoc de Xinye Chen (NumPEX)

postdoc avec LOCEAN

Accurate floating-point summation algorithm: Condense & Distill

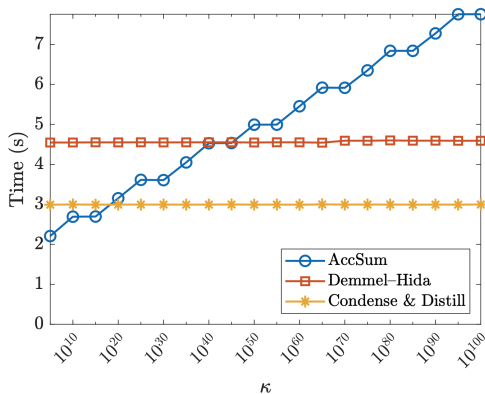
Distillation methods

- transform an ill-conditioned sum into an equivalent well-conditioned one
- expensive 😞

$a \oplus b$ exact when a and b have the same exponent and the same least significant bit

Condense & Distill

- transforms the sum into a smaller sum (condensation) which can be distilled inexpensively
- runs in the working precision
- cost independent of the conditioning
- near perfect parallel scaling



[SISC 2025]

Computation of $\sum_{i=1}^n x_i$ with $n = 10^8$

Probabilistic estimation of the accuracy of inner products $s = x^T y$ ($x, y \in \mathbb{R}^n$)

Numerical validation using stochastic arithmetic

- requires the computation of d inner products with stochastic rounding
- prevents the use of optimized libraries which do not support stochastic rounding modes.

⇒ **major performance hurdle**

We propose 2 methods to get a **reliable accuracy estimation** by **standard deterministic inner products**.

randomness introduced to estimate the accuracy:

- random perturbations to the **input x and/or y**
- or random perturbations to the **output $\hat{s}$** .

⇒ **no intrusive modifications** in the intermediate computations of the inner products

⇒ can rely on **optimized implementations** such as the BLAS.

Method 1:

We randomly perturb each representative of x and compute

$$s^{(1)} = (x^{(1)} + \Delta x^{(1)})^T y, \quad |\Delta x^{(1)}| \leq \delta |x^{(1)}|,$$

...

$$s^{(d)} = (x^{(d)} + \Delta x^{(d)})^T y, \quad |\Delta x^{(d)}| \leq \delta |x^{(d)}|,$$

where $\Delta x^{(1)}, \dots, \Delta x^{(d)}$ are random perturbations.

the perturbed representatives of x differ by a factor of order δ
 $\Rightarrow$ the $s^{(i)}$ will differ by a factor of order $\kappa\delta$.

Method 1 **implicitly** estimates the condition number κ .

Method 2:

We compute a deterministic inner product $\hat{s}$.

Then we compute $\hat{\kappa}$, an explicit estimation of the condition number $\kappa := \frac{|x|^T |y|}{|x^T y|}$.

Randomness introduced in the d representatives of s :

$$s^{(i)} = \hat{s} + \Delta s^{(i)}, \quad i = 1:d,$$

where $|\Delta s^{(i)}| \approx \delta \hat{\kappa} |\hat{s}|$, δ controlling the size of the perturbations.

Method 2 **explicitly** estimates κ , and **randomizes the output** with a perturbation of size $\kappa\delta$.

Reliability of the estimators

- **probabilistic analysis** proves both methods to be **reliable estimators**
- reliability also **confirmed via experiments**: both methods compute estimations comparable to the stochastic validation method implemented in CADNA

We compute in double precision 200 pairs of vectors x and y of size $n = 100$.

To simulate previous errors, we generate a triplet of perturbed vectors

$$\mathbf{x} = \left(x^{(1)}, x^{(2)}, x^{(3)} \right) = \left(x + \Delta x^{(1)}, x + \Delta x^{(2)}, x + \Delta x^{(3)} \right)$$

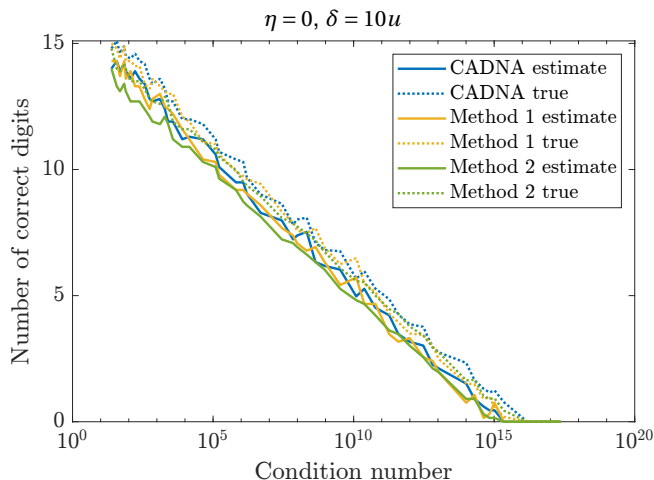
where $|\Delta x^{(j)}| \leq \eta |x|$.

η represents the noise affecting the vectors

if $\eta = 0$ the vectors are exact $\Rightarrow$ randomness must be added

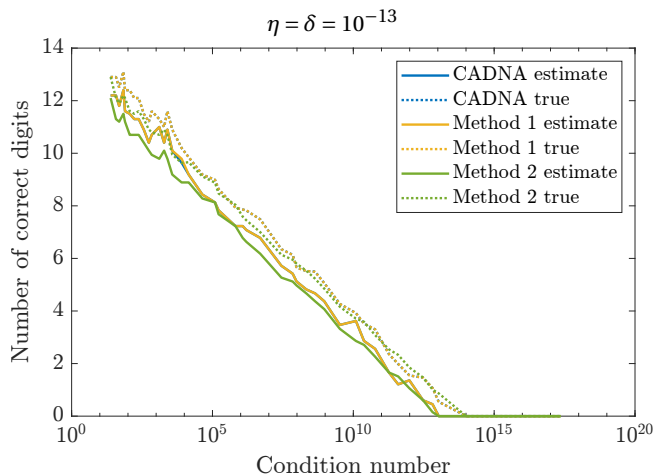
For CADNA, Method 1, and Method 2, we report:

- the *estimated* accuracy provided by the method
- the *true* accuracy obtained by comparing the computed result to the correctly rounded result.



$\delta = 10u$ appears to be a suitable choice.

Method 1 and Method 2 compute a result with comparable accuracy to CADNA, while providing a reasonably tight estimate of their accuracy.



Method 1 equivalent to CADNA

Method 2 slightly pessimistic

Which of Method 1 or Method 2 should be preferred?

- Method 2 tends to be slightly more pessimistic than Method 1.
- In terms of cost,
 - Method 1 computes d inner products (in practice $d = 3$), whereas Method 2 only computes 2 of them.
 - when the input vectors are already affected by stochastic perturbations, Method 2 also requires to measure the noise δ .

Recommendation

- Method 2 when the input vectors are exact and performance is paramount
- Method 1 when the input vectors are already perturbed and/or a very tight estimate is desired.

inspired from FP-ANR format

representation with a number and the position of the 1st erroneous bit

Research directions:

- conversions FP-INT to/from classic intervals
- development of a new interval library

Internship Yizhi Yang

Sur les algorithmes en précision mixte :

Nicholas Higham, Theo Mary,
Mixed precision algorithms in numerical linear algebra,
Acta Numerica, 31:347–414 (2022).

<https://hal.science/hal-03537373>

Sur l'arrondi stochastique :

Matteo Croci, Massimiliano Fasi, Nicholas Higham, Theo Mary, Mantas Mikaitis,
Stochastic Rounding: Implementation, Error Analysis, and Applications,
Roy. Soc. Open Sci., 9(3), 1–25 (2022).

<https://hal.science/hal-03378080>

SpMV en précision adaptative :

 S. Graillat, F. Jézéquel, T. Mary, R. Molina, Adaptive precision sparse matrix-vector product and its application to Krylov solvers, SIAM Journal on Scientific Computing, 46(1), pages C30-C56, 2024.

<https://hal.science/hal-03561193>

 S. Graillat, F. Jézéquel, T. Mary, R. Molina, D. Mukunoki, Reduced-Precision and Reduced-Exponent Formats for Adaptive-Precision Sparse Matrix-Vector Product, Euro-Par 2024, Madrid, August 2024.

<https://hal.science/hal-04519666>

Auto-ajustement de la précision :

 Q. Ferro, S. Graillat, T. Hilaire, F. Jézéquel, B. Lewandowski, Neural Network Precision Tuning Using Stochastic Arithmetic, 15th Int. Workshop on Numerical Software Verification, 2022.

<https://hal.science/hal-03682645>

 Q. Ferro, S. Graillat, T. Hilaire, F. Jézéquel, Performance of precision auto-tuned neural networks, POAT-2023, within MCSoc-2023.

<https://hal.science/hal-04149501>

Condens & Distill :

 S. Graillat, T. Mary, Condense and Distill: fast distillation of large floating-point sums via condensation. SIAM Journal on Scientific Computing, In press.

<https://hal.science/hal-04507609>

Estimation probabiliste de la précision des produits scalaires :

 F. Jézéquel, T. Mary, Probabilistic estimation of the accuracy of inner products and application to stochastic validation, 2024.

<https://hal.science/hal-04554459>

Surveys :

 Nicholas Higham, Theo Mary, Mixed precision algorithms in numerical linear algebra, Acta Numerica, 31:347–414 (2022).

<https://hal.science/hal-03537373>

 Matteo Croci, Massimiliano Fasi, Nicholas Higham, Theo Mary, Mantas Mikaitis, Stochastic Rounding: Implementation, Error Analysis, and Applications, Roy. Soc. Open Sci., 9(3), 1–25 (2022).

<https://hal.science/hal-03378080>

Thank you for your attention!
Any question?